

ELEC97077: Smart Grid Tech Coursework on Data Science and Machine Learning in Power System

Presented by Dr.Wangkun Xu
wangkun.xu18@imperial.ac.uk

Nov. 2025

Coursework Overview

This is the coursework for ELEC97077—Smart Grid Technology. The purpose of this coursework is not to develop highly complex algorithms, but rather to *evaluate your understanding of how to construct an effective machine learning pipeline*. The coursework consists of two parts.

Part One. You will use a state-of-the-art *time series foundation model (TSFM)* from Amazon Science, namely *Chronos*, to perform wind power forecasting in Finland.

1. The aim is for you to gain practical familiarity with a complete ML pipeline: data collection, preprocessing, visualization, model selection, evaluation, etc. You will also observe the differences between auto-regressive and context-enhanced forecasting.
2. This is a realistic scenario, as you will collect real-world wind generation and weather data in Finland.
3. Since *Chronos* provides a Python API only, you must use **Python** for this part.

Part Two. You will work with power system *state estimation* and *bad data detection (BDD)*.

1. A dataset of sensor measurements (generated using DC-OPF under realistic loads) will be provided.
2. There are four tasks. The first three assess your understanding of state estimation and BDD. The last two tasks involve basic ML development, where you will design ML model(s) to detect FDI attacks.
3. You may use either **Matlab** or **Python** for this part. However, the data will be provided in **.npz** format, and your submission *must also* be in **.npz** format.
4. You are strongly encouraged to use ML libraries such as Matlab's Statistics and Machine Learning Toolbox¹ or Python's scikit-learn².

Before you start, please note,

1. You will work individually.

¹<https://uk.mathworks.com/products/statistics.html>

²<https://scikit-learn.org/stable/>

2. For all tasks, you must provide numerical results *and* a brief description of your methodology (for example pseudo-code, mathematical expressions, *or* text explanation). Some tasks may require knowledge slightly beyond the lecture notes; you are expected to consult external sources where necessary.
3. For any questions, please contact Dr. Wangkun Xu via MS Teams or email.
4. Submit your report and forecasting result by emailing Wangkun Xu. *Submission format*: submit one `<surname>_<CID>.pdf` file and one `<surname>_<CID>.npy` file. Also include the full name and CID in the report. The report *must not* exceed 3 pages (extra pages will not be marked). Code can be placed in an appendix (not included in the 3-page limit).
5. Python users are strongly encouraged to use Google Colab to avoid dependency configuration on local machines.
6. *Policy on using AI*: AI tools (e.g. ChatGPT) are permitted. However, you must declare any usage at the end of the report.

Part One: Wind Generation Forecasting using TSFM [33]

Task One: Get Familiar with Chronos [14]

Go to <https://github.com/amazon-science/chronos-forecasting> and locate the example titled “A minimal example showing how to perform forecasting using Chronos-2”.

Re-implement this example in Google Colab: <https://colab.research.google.com/>. Your Colab notebook should run successfully and produce the forecast plot.

1. In your own words, explain the concept of zero-shot learning in the context of time-series forecasting [2].
2. Print the dataframes `context_df` and `future_df`. Explain [4]:
 - Based on their columns, identify which column(s) are used as features (covariates or contexts) and which column is the forecasting target.
 - How is the sizes of the two datasets?
3. In `pipeline.predict_df`, the argument `quantile_levels` specifies a set of quantiles. By inspecting the forecast visualization, explain what these quantile levels represent in the forecast results [2].
4. Chronos-2 supports both auto-regressive forecasting and context/feature/covariate-enhanced forecasting. Using the function call `pipeline.predict_df` and the contents of `context_df` and `future_df`, explain [6]:
 - what auto-regression means in this setting
 - what context-enhanced forecasting means
 - how Chronos-2 switches between these two behaviors depending on the provided data

Task Two: Data Collection and Preprocessing for Wind Dataset [9]

1. Visit the FinGrid dataset: <https://data.fingrid.fi/en/datasets/75>. Download the wind power time series for the full year 2017 by setting:
 - Start time: 01/01/2017 00:00

- End time: 01/01/2018 00:00

How many rows (data points) are contained in the downloaded dataset? [2]

2. Compute the time differences between consecutive timestamps (i.e., the time delta between each row). Summarize the distribution of these deltas. Does the dataset have a regular sampling frequency? Explain briefly. [2]
3. Resample the dataset to an evenly spaced time index with **1-hour resolution**. You may use `pandas.DataFrame.resample` combined with `interpolate` if needed to fill values. After resampling and interpolation, what is the total number of rows in the resulting dataset? [5]

Task Three: Zero-shot Forecasting [10]

1. Construct the dataset that are compatible with Chronos. The dataset should contain the columns `id`, `timestamp`, and `target` (wind generation). Briefly describe how you constructed the dataset. [2]
2. You are asked to forecast the wind generation on the day 2017-10-20 (from 2017-10-20 00:00:00 to 2017-10-20 23:00:00, i.e. 24 hours in total). Use the previous month of data (672 samples) as the input history. Construct `context_df` to contain these 672 samples, and construct `future_df` to contain the 24 samples of the target day. Use the Chronos model "amazon/chronos-2" to forecast the `future_df` using `context_df` [8].
 - Plot the forecasting result in a similar style to the Chronos example on GitHub.
 - Compute the mean absolute percentage error (MAPE) of the forecast using the 50% quantile (i.e., the median forecast) and also compute the average width of the uncertainty interval between the 10% and 90% quantiles.
 - Report any findings and what can you do to improve the forecasting accuracy?

Part Two: Detecting on FDI Attacks [67]

Go to the website ³ and download the `train_dataset.npy` and `test_feature.npy`.

In the `train_dataset.npy` containing both the training feature and labels. The training features are the measurements (in per unit) generated by simulating the DC-OPF on the IEEE bus-14 system under realistic load and renewable profiles. There are three classes (labels), including

- 0: for normal measurements;
- 1: for FDI attack measurements;
- 2: for random attack measurements.

All the three classes of data have *equal* size. Load the dataset to your working space and answer the following questions based on `dataset_train`.

³https://imperiallondon-my.sharepoint.com/:f:/g/personal/wx3418_ic_ac_uk/EjdeGcWBEsR0q54_YNev3G0BE_vM SGzSFURA8ickF7QTfw?e=n87Ec5

Task One: Get Familiar with the Dataset and the Power Grid [9]

1. What is the size of the training dataset? How many samples do you have for each class and how many sensor measurements are collected? [2]
2. Referring to the IEEE bus-14 description⁴, the topology of the bus-14 system can be configured from the **branch data** where the **fbus** and **tbus** represent the from-side and to-side bus index for each transmission line. Based on the information, construct the incidence matrix $\mathbf{A}$ of bus-14 system. [3]

*Hint: you must construct the incidence matrix following the **same** sequence in the website. For example, the first row of your incidence matrix has 1 at the first element and -1 at the second element.*

3. Referring to the IEEE bus-14 description, the line susceptance $\mathbf{b}$ can be calculated as $1/\mathbf{x}$. Based on the information, calculate the branch susceptance matrix $\mathbf{B}_{pf} = \text{diag}(\mathbf{b})\mathbf{A}$ and the bus susceptance matrix $\mathbf{B}_{pi} = \mathbf{A}^T \text{diag}(\mathbf{b})\mathbf{A}$. Observing the structure of the $\mathbf{B}_{pi}$ matrix, what can you find? [4]

Task Two: State Estimation [8]

The measurements are composed of branch power flow $\mathbf{P}_{pf} \in \mathbb{R}^{20}$ for all 20 branches and bus power injection $\mathbf{P}_{pi} \in \mathbb{R}^{14}$ for all 14 buses. For each sample, they are packed as a vector $[\mathbf{P}_{pf}^T, \mathbf{P}_{pi}^T]^T \in \mathbb{R}^{34}$.

It is assumed that the measurement noise on power flow follows independent normal distribution with mean $\mu_{pf} = 0$ and standard deviation (std) $\sigma_{pf} = 0.02$. The measurement noise on power injection follows independent normal distribution with mean $\mu_{pi} = 0$ and std $\sigma_{pi} = 0.01$. The first bus is the reference bus with bus angle equal to $\theta_{ref} = 0$.

Based on the information above, answer the following questions:

1. Construct the covariance matrix $\mathbf{R}$ for the measurement. *Hint: your covariance matrix should be diagonal of size 34×34 .* [2]
2. Brief describe what is state estimation in power system and why it is important to power system operation. [3]
3. For **train_dataset**, do state estimation for class 0, class 1, and class 2, individually, to get the bus angles using the weighted least square (WLS) approach. Report the formula for state estimation and briefly discuss how you did it. [3]

Hint: remember to remove the column of the reference bus in the measurement matrix.

Task Three: Bad Data Detection [20]

1. Calculate the **residual vectors** for the measurements in **train_dataset**, based on the bus angles you calculated in the previous task. Report the formula and briefly discuss how you did it. [4]
2. Define what is true positive rate (TPR), false positive rate (FPR), true negative rate (TNR), and false negative rate (FNR) **for detecting of attacks in your own words**. [4]
3. Assume that the system operator wants to control the false positive rate on the normal measurements to be around 5.0%. Calculate the BDD threshold γ accordingly and describe briefly how you did it. [5]

*Hint: On the normal measurement, the residual follows the χ^2 distribution approximately. First, you need to determine what is the ‘degree of freedom’ is in this problem. Then use Matlab’s **chi2inv***

⁴<https://github.com/MATPOWER/matpower/blob/master/data/case14.m>

or Python SciPy's `scipy.stats.chi2.ppf` to find the inverse cdf as the threshold. Search online for extra information.

4. **Before calculate the residuals, give a guess on** the alarm rates on normal, random attack, and FDI attack datasets, based on your knowledge from the lecture and explain why. Then for each class, calculate the residuals on the datasets. What are the alarm rates for these dataset and do they follow your intuition? You need to briefly discuss on how you achieve this. [7]

Task Four: Detecting FDI Attacks using Machine Learning [30]

In this task, you need to design machine learning algorithm to distinguish between **normal** measurement and **FDI attack** measurement. You need to train and tune your ML detector on the `train_dataset`. Finally, report your predict label for the measurement in `test_feature`.

1. Using **different** machine learning techniques to train a detector to classify the normal measurement and FDI attacks on the `train_dataset`. [15/100]

Note that

- There is no restrictions on what machine learning techniques you use. It is suggested to start from the classic ones that are covered in the lecture, such as linear support vector machine, nonlinear (kernel) support vector machine, decision trees, etc. It is also recommended to use Matlab's build-in functions or Python's scikit-learn package.
 - Briefly describe how you develop your machine learning models and how you choose the hyper-parameters.
 - Report the performance of your classifier in the format of TPR, FPR, and F1 score. Compare different methods and discuss any findings you have.
2. Report your detection results on the `test_feature`. Note that you can only report the results from **one** detector you believe to have the best performance. You must report it in `<surname>_<cid>.npz` format. Use the same convention, i.e., class 0 for normal measurement and class 1 for FDI attack measurement. [15/100]

Note that

- You are reporting a list of 0 or 1.
- Your results will be assessed by the F1 scored.

For Python user, you can save your prediction by (for reference only)

```
np.save(f"<surbane>_<cid>.npz", y_pred)
```

To make sure that your submission can be assessed, use the following code to see if you have a legit result (for reference only)

```
def assess(y_pred):
    assert np.all((y_pred == 0) | (y_pred == 1))
    assert len(y_pred.shape) == 1
    assert y_pred.shape[0] == 1200
```